Pandemic District & State E-Pass – Copilot Prompt

Before you paste

- The element IDs, error messages and dropdown options were read from the live pages and PandemicJS.js ahead of time. The prompt tells the AI to verify each one, so any change on the day gets corrected and recorded.
- If you already know the workspace stack, replace the TECH STACK block with one line (e.g. "Selenium + Java + TestNG").
- Confirm the CSV columns after Precondition; they were cut off in the screenshot. The prompt assumes the same 12 columns as the other assignments.
- Evaluate plan (3 attempts): ~50% once the analysis file and testcases.csv exist; ~90% after the first full test run; final after defects/defect_report.json is written. Turn each Evaluate's feedback into a short follow-up prompt.
- If Evaluate says the format-gap tests should assert the documented behaviour (accepted) instead of rejection, send a refinement prompt to flip those assertions and keep logging each acceptance as a defect.

Prompt

```
ROLE
Act as a senior QA automation engineer and SDET working inside the current VS Code workspace. You are completing an AI-assisted UI automation assessment for the District E-Pass and State E-Pass modules of the Pandemic District & State E-Pass web application. Create the files in the workspace, run them, and refine; do not just print file contents in chat. Address exactly the stated requirements: nothing more, nothing less. Be token-efficient: no long explanations, no extra files.


TECH STACK
Use the stack already set up in the workspace: pom.xml -> Selenium + Java (TestNG/JUnit as configured), run with mvn test; package.json with @playwright/test -> Playwright + TypeScript, run with npx playwright test. If neither is set up, use Playwright + TypeScript. Page Object Model, explicit waits, assertions. Run everything from the VS Code integrated terminal.


WORKSPACE RULES
- First read the workspace, especially User Stories.txt (source of truth for District US-01 to US-09, State US-01 to US-12, and the URLs).
- Do NOT change, rename, delete, or restructure anything provided. Build on top of it only.
- No redundant files or folders. Follow standard naming conventions.


APPLICATION UNDER TEST
District E-Pass: https://webapps.tekstac.com/SeleniumApp2/Pandemic/DistrictPass.html (District US-01 to US-09)
State E-Pass: https://webapps.tekstac.com/SeleniumApp2/Pandemic/StatePass.html (State US-01 to US-12)
No login. Navigate directly to each form's URL before every test.
Both forms are client-side only (static HTML + vanilla JS in the shared PandemicJS.js; no backend, no persistence across reload). A successful submission hides the form and shows a "Submitted Details" table on the same page; there is no reference/application ID.
User story IDs restart per module: always write them as "District US-01" / "State US-01".


PRE-INSPECTED ELEMENT IDs (from my review of the pages and PandemicJS.js; verify each against the live DOM and record any correction)
```

District (validated by districtPassSubmit()):
- Name: name -> nameError "Please Enter Your Name"
- Email: email -> emailError "Please Enter Your Email Id"
- Mobile: mobile1 / mobile2 / mobile3 (maxlength 3/3/4) -> mobileError "Please Enter Your Mobile number"
- District: district (select; Chennai, Coimbatore, Madurai, Trichy, Salem, Tirunelveli) -> districtError "Please select your District"
- Gender: radios name="Gender" (Male / Female / Others) -> genderError "Please select gender"
- Aadhaar: aadhaar -> aadhaarError "Please Enter Your Adhaar No."
- Reason: reason (select; Wedding, Vaccination, Interview, Funeral, Exams) -> reasonError "Please Enter Your Reason"
- On success: districtForm hidden, districtView shown; cells nameDisplay, emailDisplay, mobileDisplay, districtDisplay, genderDisplay, aadhaarDisplay, reasonDisplay

State (validated by stateSubmit()):
- Name: nameState -> nameStateError "Please Enter Your Name"
- Email: emailState -> emailStateError "Please Enter Your Email Id"
- Mobile: mobile1state / mobile2state / mobile3state (maxlength 3/3/4) -> mobileStateError "Please Enter Your Mobile number"
- State: state (select; Maharashtra, Goa, Kerala, Gujarat, Punjab, Delhi) -> stateError "Please select state"
- Gender: radios name="GenderState" (Male / Female / Others) -> genderStateError "Please select gender"
- Aadhaar: aadhaarState -> aadhaarStateError "Please Enter Your Adhaar No."
- Reason: reasonState (select; Wedding, Vaccination, Interview, Funeral, Exams) -> reasonStateError "Please select Reason"
- Travel By: travelBy (select; Bus, Train, Car) -> modeError "Please select your mode of travel"
- Travel Date: date -> dateError "Please Enter date"
- Anything to Say: optional textarea (find its id)
- On success: stateForm hidden, stateView shown; cells statenameDisplay, stateemailDisplay, statemobileDisplay, stateDisplay, stategenderDisplay, stateaadhaarDisplay, statereasonDisplay, statetravelDisplay, statedateDisplay

Still to find: submit button IDs, individual radio IDs, the textarea ID, and the value attribute of each "-----Select-----" option (the JS compares dropdowns to "select").

VALIDATION RULES (from the brief)

District (all 7 fields mandatory; no optional field):
- Name: non-empty text; no letters-only restriction enforced
- Email: non-empty text; NOT format-validated (no "@"/domain check)
- Mobile Number: three segments, each non-empty (maxlength 3/3/4); no digit-only enforcement
- District: must select an option other than "-----Select-----"
- Gender: one radio option (Male / Female / Others) must be selected
- Aadhaar Number: non-empty text; NOT length/digit-format-validated
- Reason: must select an option other than "-----Select-----"

State (9 of 10 fields mandatory; all except Anything to Say):
- Name: non-empty text
- Email: non-empty text; NOT format-validated
- Mobile Number: three segments, each non-empty (maxlength 3/3/4); no digit-only enforcement
- State: must select an option other than "-----Select-----"
- Gender: one radio option must be selected
- Aadhaar Number: non-empty text; NOT length/digit-format-validated
- Reason, Travel By: must select an option other than "-----Select-----"
- Travel Date: non-empty text; NOT date-format-validated

- Anything to Say: optional textarea

HOW TO TREAT THE "NOT ENFORCED" RULES

PandemicJS.js only checks for empty values, the "select" default, and an unchecked gender radio. The missing checks are validation gaps: name accepting digits/symbols, email without "@" or domain, letters/symbols in mobile segments, non-numeric or wrong-length Aadhaar (valid is 12 digits), an invalid Travel Date string, and whitespace-only input (test whether " " passes the empty check). For each gap: fill every other field validly, enter the one invalid value, submit, and assert that a correctly validated E-Pass form rejects it (error shown, form still visible). When the app accepts it (form hidden, value echoed in Submitted Details), the test fails and the defect logger records it. Mandatory-field tests assert the documented error message and that the form stays visible.

STEP 1: INSPECT BOTH FORMS

Open DistrictPass.html, StatePass.html, and PandemicJS.js (from the page's script tag). Verify every ID above, find the missing ones, and record field types, maxlength, option values, and where each error label renders. Submit an empty form, then one empty field at a time, then the invalid values above, and record what actually happens. Read districtPassSubmit() and stateSubmit() to confirm the real validation logic rather than guessing from the UI.

STEP 2: ACCEPTANCE CRITERIA

Write acceptance criteria for District US-01 to US-09 and State US-01 to US-12 from User Stories.txt plus observed behaviour.

STEP 3: VALIDATION RULES

Document all field-level rules for both modules, including the rules the application does NOT enforce (email format, numeric mobile/Aadhaar, date format on State Pass), each confirmed against PandemicJS.js.

Save Steps 1-3 in: Application Analysis & Requirements.txt (overview, navigation, element ID map incl. error labels and result cells, field types, acceptance criteria, validation rules, not-enforced rules, observed behaviour, discrepancies) for both modules.

STEP 4: TEST CASES (CSV)

Using the analysis file as context, create testcases.csv with 40 to 45 test cases covering all 21 user stories: District about 20 (US-01 to US-09), State about 20-25 (US-01 to US-12).

Columns exactly as in the brief: TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Distribution: 60-70% Negative/Validation (Critical/High): empty form, each mandatory field empty, each dropdown left at default, no gender, format gaps. Boundary (Medium): empty vs partially filled mobile segments, segment maxlength, non-numeric Aadhaar, invalid Date string on State Pass. Positive (Low): valid submission shows correct Submitted Details; State submits with Anything to Say left blank.

After execution, fill Actual_Result and Status from real results.

STEP 5: AUTOMATION

- Browser setup/teardown (base class or config) with explicit waits
- Page objects: DistrictPassPage and StatePassPage, using only the verified IDs, with methods to fill each field, select options, choose gender, submit, read each error label, check form/result visibility, and read result cells
- Reusable helpers: test data with one valid baseline per module; fillAllValidExcept(field, value) on each page object; a DefectLogger utility that records module, user story, test case ID, entered data, observed behaviour, expected result, severity and priority from failing validation checks and writes the consolidated defects/defect_report.json

- At least 30 test methods with assertions, at least 15 per module, prioritising Critical/High validations over happy paths
- Each test navigates directly to its form URL; no login helper
- For maxlength checks, type character by character and assert the resulting value length

STEP 6: RUN AND REFINE

Run the suite from the terminal. Classify each failure: locator, timing, wrong assertion, or genuine validation defect. Fix automation issues and re-run. Change an assertion only if the brief or PandemicJS.js proves it wrong. Never weaken a valid validation assertion to make a test pass. Re-run the full suite at the end and keep the report as execution evidence.

STEP 7: DEFECT REPORT

Save all defects in one file: defects/defect_report.json, with at least 8 validation defects and at least 4 per module, taken only from actual execution. Never invent defects; if fewer are found, run more negative/boundary tests.

Each defect: defect_id, module, user_story_id, test_case_id, title, steps_to_reproduce, entered_data (what was entered), observed_behavior (what happened), expected_result, severity, priority, status, evidence (test name / screenshot path).

FINAL CHECK

- Analysis file covering both modules: element IDs, field types, acceptance criteria for all 21 stories, validation rules incl. not-enforced rules
- testcases.csv with 40-45 cases, all 21 stories, required columns, 60-70% negative
- DistrictPassPage and StatePassPage, fillAllValidExcept helper, DefectLogger; 30+ tests (15+ per module) executed
- defects/defect_report.json with 8+ real defects (4+ per module) incl. entered data, observed behaviour, expected result, severity, priority
- Workspace structure unchanged, no redundant files, all files saved (Ctrl+S)

Do the work; do not just describe it. Start by reading the workspace and inspecting both live forms and PandemicJS.js.